Evaluation of Model Degradation in PaddleOCR, UltOCR, and TrOCR Across Baseline and TensorFlow Lite Environments

Rameel Ahmed
Department of Mechatronics Engineering, College of E&ME, National University of Sciences and Technology,
Islamabad, Pakistan
rahmed.mts43ceme@student.nust.edu.pk

Noman Shabbir
Department of Mechatronics Engineering, College of E&ME, National University of Sciences and Technology,
Islamabad, Pakistan
nshabbir.mts43ceme@student.nust.edu.pk

Muhammad Wasif Raza
Department of Mechatronics Engineering, College of E&ME, National University of Sciences and Technology,
Islamabad, Pakistan
mwasif.mts43ceme@student.nust.edu.pk

Ayesha Zeb
Department of Mechatronics Engineering, College of E&ME, National University of Sciences and Technology,
Islamabad, Pakistan
ayesha.zeb@ceme.nust.edu.pk

Hassan Elahi
Department of Mechatronics Engineering, College of E&ME, National University of Sciences and Technology,
Islamabad, Pakistan
hassan.elahi@ceme.nust.edu.pk

2024 International Conference on Robotics and Automation in Industry (ICRAI) | 979-8-3503-5570-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICRAI62391.2024.10894257

***Abstract*—Optical Character Recognition (OCR) models are deployed on edge devices for many applications ranging from document scanning to real-time text recognition. For edge deployment, it is suitable to convert OCR models to TensorFlow Lite (TFLite) format. However, there is uncertainty of model's performance degradation after conversion. This study evaluates three prominent OCR models—PaddleOCR (CNN-RNN architecture), UltOCR (MASTER architecture), and TrOCR (Transformer-based architecture)—selected for their distinct architectures. Baseline, TFLite 32-bit, and quantized 16-bit versions of these models were tested on datasets containing words, numbers, and sentences to identify which architecture maintains the performance after conversion. The findings indicate that PaddleOCR and UltOCR achieved minimal degradation. However, TrOCR experienced a significant accuracy loss of 6.81% post-conversion in the sentences category. The study provides valuable insights in selecting which OCR architecture should be used for edge deployment.**

Keywords—OCR, PaddleOCR, UltOCR, transformer, TrOCR, TensorFlow Lite, quantization

I. INTRODUCTION

Optical Character Recognition (OCR) technology has revolutionized digitization across industries, from facilitating automation in healthcare [1] to enabling real-time translation, the utility of OCR is widespread. With the growth of OCR technologies, modern machine learning architectures like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and Transformer models have noticeably improved text recognition accuracy [2].

Deploying OCR models on edge devices poses challenges due to limitations of computational power, memory, and energy consumption. These devices require models that not only deliver high performance but perform efficiently under resource-limited conditions. TensorFlow Lite (TFLite) offers a solution by converting complex models into optimized lightweight versions, suitable for real-time applications on such devices [3]. However, it is difficult not to be concerned about performance issues after converting to TFLite.

Various studies have deployed OCR models on edge devices. For instance, Pandey et al. [4] implemented a CNN architecture for the MNIST database on a Raspberry Pi 4 Model B using the TensorFlow Lite framework, focusing solely on the classification of handwritten digits. Similarly, Revanth et al. [5] developed the TFlite version of the CNN architecture model for character-level recognition. Verma et al. [6] provided a detailed comparison of TensorFlow-TensorRT (TF-TRT) and TFLite inference compilers for image classification models and object detection models. Additionally, TFLite conversion of deep learning models focusing on Human Activity Recognition (HAR) has been explored [7]. Despite the growing use of TFLite conversions, there are few performance evaluations of prominent OCR architectures in the TFLite environment. Bolkisev and Ivanov [8] demonstrated how model quantization can be opted to deploy OCR on low-end hardware without significant performance loss.

Even with improvements in OCR technologies, a major challenge that remains is model degradation after conversion and quantization for edge deployment, because it reduces the model size compromising the accuracy [9]. It is necessary to understand which specific OCR architectures better withstand the conversion process in order to be deployed on resource-constrained devices.

This study aims to analyze the performance consistency of three prominent OCR architectures—PaddleOCR, TrOCR, and UltOCR— in Baseline form, TFLite, and Float16 quantization (which further optimized the model size and inference time) for deployment on resource-constrained environments. By assessing factors such as accuracy and model size, this study provides valuable insights into the practicality of choosing between OCR architectures suitable for deployment on edge devices.

II. METHODOLOGY

A. Architectures and Conversion Technique

PaddleOCR is an advanced system developed for text recognition. The recognition module uses the CRNN architecture with Connectionist Temporal Classification (CTC) for effective text decoding [10]. PP-OCRv3 version of PaddleOCR is used, which is an English text recognition model incorporating several optimizations including the lightweight SVTR_LCNet architecture improving both speed and accuracy. Techniques like text context augmentation for data augmentation, text rotation network for self-supervised pre-training, and methods like unified deep mutual learning are used, all of which improve the model's speed and accuracy in text recognition. PaddleOCR architecture is visualized in Fig. 1.

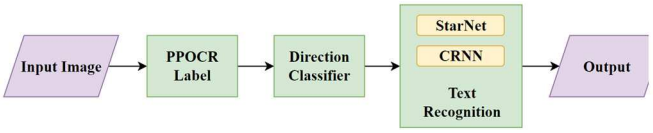

Fig. 1. Architecture of PaddleOCR based on PP-OCRv3.

TrOCR utilizes the transformer architecture [11], bypassing traditional CNN-based OCR systems as shown in Fig. 2. In the model, input images first undergo a resizing process to have a uniform size of 384×384 pixels. After which they are divided into patches of size 16×16 pixels. These patches are passed to an image transformer playing the role of the encoder that employs self-attention techniques to assess and represent the image information visually. Similar to Vision Transformer (ViT) [12] designs, the encoder is improved in its ability to understand image contexts when initialized with the pre-trained weights such as from ViT or Data-Efficient Image Transformer (DeiT) [13]. The output produced by the encoder is then fed into a transformer-based decoder similar to Bidirectional Encoder Representations from Transformers (BERT) models [14], which excel at text generation without reinforcement by an extra language model. This architecture allows TrOCR to perform end-to-end text recognition efficiently [15].

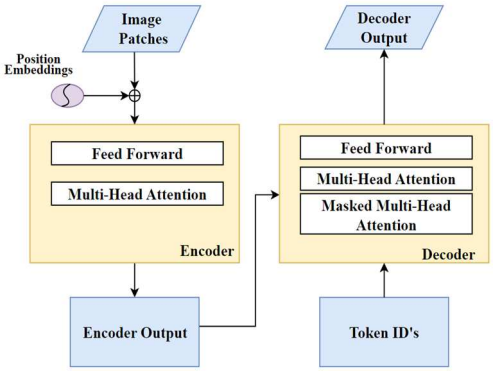

Fig. 2. TrOCR, a transformer based architecture containing encoder and decoder.

UltOCR downloaded from the Python Package Index (PyPI) [16] uses the multi-aspect non-local network for scene text recognition (MASTER) framework [17] as shown in Fig. 3. It begins with resizing and conversion of input images into patches which are then processed by an encoder featuring a multi-aspect global context attention mechanism. The encoder captures comprehensive image contexts, enhancing the model's ability to recognize text in various orientations and styles. The encoder's output is decoded by a transformer-based decoder with masked multi-head attention for accurately predicting character sequences.

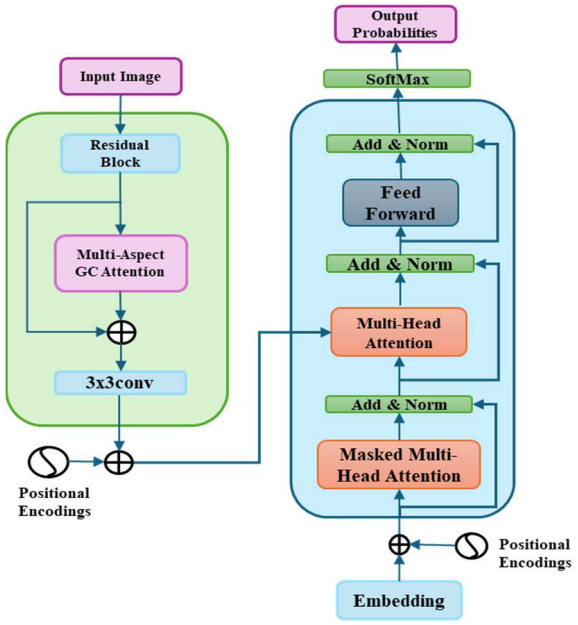

Fig. 3. UltOCR architecture with multi-aspect global context attention mechanism.

B. Conversion and Quantization Process

OCR models were initially converted into the Open Neural Network Exchange (ONNX) format for compatibility with the TensorFlow framework. The input and output shapes were defined for this conversion to indicate the size and structure of data that the model can accept and produce. Fixed shapes allow for efficient memory allocation and perform better under the optimization techniques in the conversion and quantization processes.

The ONNX model was converted to TensorFlow format(version 2.8.0) and then to TFLite 32-bit. TFlite models are optimized for performance, the conversion fixed the input and output dimensions for resource utilization on edge devices by doing an appropriate flattening. Float16 quantization was applied after TFLite conversion to reduce model size without losing model accuracy. Quantization transforms the model's weights and activations from higher precision (Float32) to lower precision (Float16), causing changes in the models' performance.

C. Performance Evaluation Metrics

Levenshtein Distance measures the minimum number of single-character edits required to transform the predicted text into the reference text [18]. This metric is useful for evaluating both numbers and short words since it quantifies small, local errors that may go unnoticed by word-level metrics.

Character Error Rate (CER) [19] is defined as follows:

$$CER = \frac{\square+\square+\square}{\square} \quad (1)$$

Where S is the number of substitutions, D is deletions, I is insertions, and N is the total number of characters in the reference text. CER is particularly useful when evaluating the performance of OCR models on shorter text sequences or when

working with languages that do not have clear word boundaries. Lower CER values indicate better performance.

Word Error Rate (WER) [20] is defined as:

$$\square\square\square = \frac{\square+\square+\square}{\square} \quad (2)$$

WER is useful in the case of sentence-level predictions, where word and space placement play an important role in the overall readability of the text. This is particularly important when comparing models that segment sentences differently.

Character Accuracy [21] is also an important metric to evaluate OCR systems, counting the number of correctly recognized characters as a percentage of the total number of characters in the reference text.

$$\text{Character Accuracy (\%)} = \left(1 - \frac{\square+\square+\square}{\square}\right) \times 100 \quad (3)$$

D. Dataset

The IIIT5K-Words dataset was chosen for word-level and numeric recognition [22]. It contains 5,000 images of scene text. In the dataset labels, all the words are capitalized. To improve accuracy, the labels were adjusted to match the case present in the images. The dataset was divided into two categories: numeric and word data, as shown in Fig. 4, to evaluate how well each model handles numbers and text individually.

For sentence-level evaluation, the text detection box dataset from Roboflow [23] was acquired. It includes 527 images of text in various contexts as illustrated in Fig. 4. YOLOv8 was used for text detection on the dataset images. The resulting cropped images were then used for performance evaluation.

Fig. 4. Sample images for Words, Numbers, and Sentences.

III. RESULTS

The performance of PaddleOCR, UltOCR, and TrOCR models were evaluated on a system with a Ryzen 5 4500U CPU, 16 Gb Ram featuring 6 cores and base frequency of 2.38 GHz. Each model was evaluated in its baseline, TFLite 32-bit, and Float16 quantized versions across the datasets.

PaddleOCR demonstrated strong performance across all datasets with minimal degradation after conversion and quantization. In the words dataset, the baseline model achieved a CER of 0.0756 and a WER of 0.3725. The TFLite and Float16 versions showed slight improvements, possibly due to optimizations during conversion, as shown in Table I. In the numbers dataset, the CER increased after conversion from 0.0053 to 0.0075, maintaining an accuracy of 99.25% in both TFLite and Float16 versions.

For sentence-level recognition, the baseline model's performance was comparatively poor, mainly due to PaddleOCR missing spaces between words, resulting in a CER of 0.2056 and a WER of 0.3605. However, there was no notable degradation in the TFLite and Float16 versions as shown in Table I. The model size was significantly reduced from 9.8 MB to 4.4 MB after quantization as illustrated in Fig. 5. The inference time decreased from approximately 20 ms to 16 ms on CPU.

TABLE I. Performance Metrics Comparison for PaddleOCR in Baseline, TFLite, and TFLite 16-bit Versions.

PaddleOCR Model	CER	WER	Levenshtein	Accuracy (%)
Baseline (Words)	0.0756	0.3725	0.3820	92.44
TFlite (Words)	0.0725	0.3658	0.3658	92.75
Float16 (Words)	0.0732	0.3624	0.3624	92.68
Baseline (Numbers)	0.0053	0.0150	0.0150	99.47
TFlite (Numbers)	0.0075	0.0150	0.220	99.25
Float16 (Numbers)	0.0075	0.0150	0.220	99.25
Baseline(Sentences)	0.2056	0.3605	1.7037	79.44
TFlite (Sentences)	0.2281	0.3663	3.0741	77.19
Float16 (Sentences)	0.2306	0.3609	3.057	76.32

UltOCR exhibited the minimum CER and WER over all datasets as reported in Table II, showing better character recognition accuracy compared to PaddleOCR and TrOCR. The baseline model showed 96.94% of accuracy in the words dataset and the performance remained the same in TFLite. The Float16 version experienced a little degradation by 1.2% decrease in accuracy. Similarly, the baseline and TFlite version obtained an accuracy of 97.52% for the numbers. The Float16 version saw a slight dip to 96.47% which is 1.07% less accurate. All three versions achieved an accuracy of more than 95% for sentence-level recognition however, there was a drop of 2.0% accuracy in the Float16 version compared to baseline and TFlite versions as shown in Table II.

As illustrated in Fig. 5, the model size of UltOCR was decreased from 254.9 MB to 117.1 MB (54%) after quantization, while it is still larger than PaddleOCR by over 51%. Furthermore, it took UltOCR approximately 2 seconds as inference time on CPU, which is 125 times slower than PaddleOCR.

TABLE II. Performance Metrics Comparison for UltOCR in Baseline, TFLite, and TFLite 16-bit Versions.

UltOCR Model	CER	WER	Levenshtein	Accuracy (%)
Baseline(Words)	0.0306	0.100	0.0906	96.94
TFlite (Words)	0.0306	0.100	0.0906	96.94
Float16 (Words)	0.0424	0.103	0.1402	95.76
Baseline (Numbers)	0.0246	0.0709	0.0709	97.52
TFlite (Numbers)	0.0246	0.0709	0.0709	97.52
Float16 (Numbers)	0.0353	0.1063	0.0992	96.47
Baseline(Sentences)	0.0262	0.0909	0.6146	97.38
TFlite (Sentences)	0.0262	0.0909	0.6146	97.38
Float16 (Sentences)	0.0458	0.1129	0.6321	95.42

In contrast, the performance of TrOCR suffered great degradation especially for the sentences dataset after conversion. The accuracy dropped by 6.81% from baseline version to TFLite version on the sentences primarily due to avoiding spaces between words and incorrect letters

predictions. There was slight degradation in WER after the Float16 quantization, as shown in Table III. Moreover, TrOCR performance remained constant on numbers with an accuracy of 92% but on words category it dropped by 1.95% after TFLite conversion.

The final model size was reduced by 60% to 106.9 MB after the quantization as shown in Fig. 5. The single image inference time was approximately 0.4 seconds. The model consistently predicted capitalized words, which decreased overall accuracy.

TABLE III. Performance Metrics Comparison for TrOCR in Baseline, TFLite, and TFLite 16-bit Versions

TrOCR Model	CER	WER	Levenshtein	Accuracy (%)
Baseline (Words)	0.077	0.2181	0.4631	92.56
TFlite (Words)	0.0965	0.3054	0.6745	90.55
Float16 (Words)	0.0965	0.3254	0.6795	90.55
Baseline (Numbers)	0.0757	0.1538	0.2172	92.43
TFlite (Numbers)	0.0787	0.1629	0.2489	92.13
Float16 (Numbers)	0.0787	0.1629	0.2489	92.13
Baseline (Sentences)	0.3515	0.7531	11.0	64.85
TFlite (Sentences)	0.4196	0.8278	12.963	58.04
Float16 (Sentences)	0.4196	0.8516	13.9	58.04

Fig. 6 represents the accuracy comparison across all the datasets for PaddleOCR, UltOCR, and TrOCR in baseline, TFlite and quantized versions. As shown, TrOCR experiences the most significant drop in accuracy, particularly in the sentences category, while UltOCR and PaddleOCR maintained relatively stable performance with smaller declines.

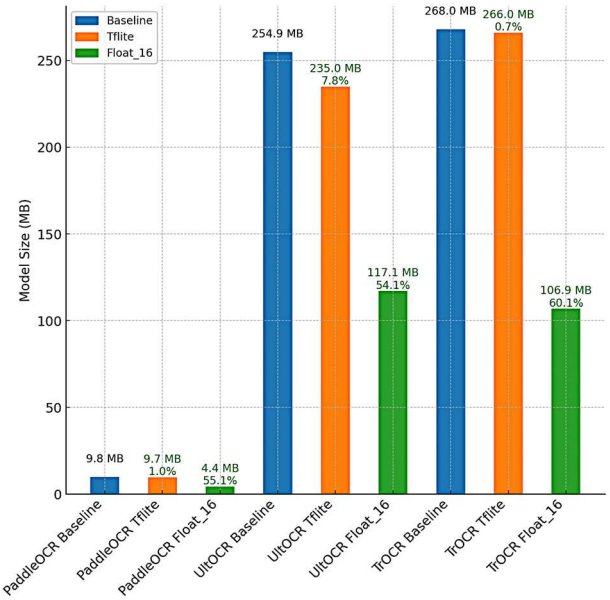

Fig. 5. Comparison of Model Sizes for PaddleOCR, UltOCR, and TrOCR across Baseline, Tflite, and Float16 variants.

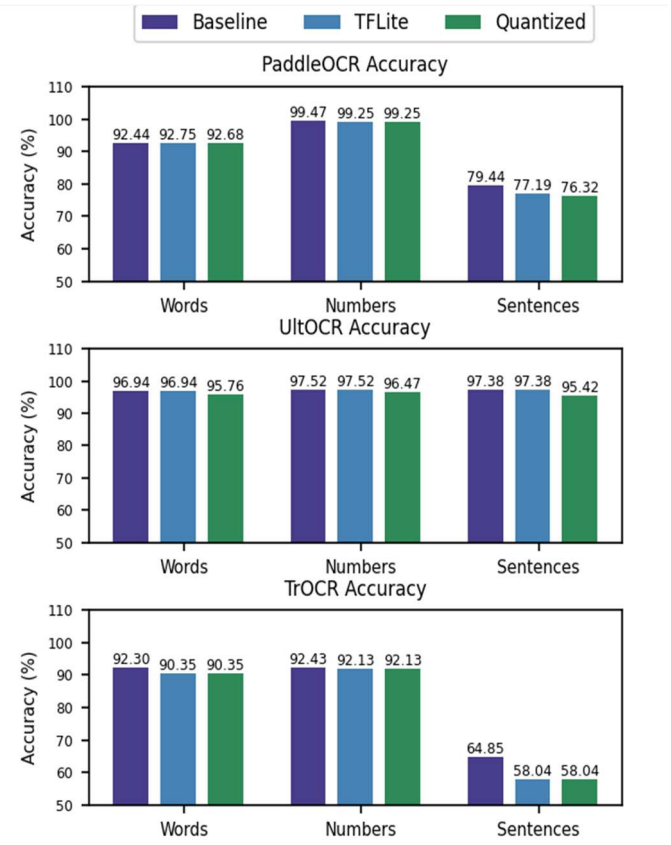

Fig. 6. Accuracy comparison of PaddleOCR, UltOCR, and TrOCR models across Words, Numbers, and Sentences categories.

IV. CONCLUSION AND FUTURE WORK

After in-depth research and evaluation, this study evaluated that PaddleOCR, with CNN-RNN based model, is most suitable for edge deployment due to its compact size of 4.4 MB in quantized form and fast inference time of 16 ms. UltOCR, based on the MASTER architecture, achieves the highest accuracy with the lowest CER and WER. However, its large model size of 117.1 MB and slower inference time of 2000 ms limit its practicality for real-time applications on resource-constrained devices. TrOCR, with the Transformer-based model, showed significant accuracy degradation after conversion.

Future research should prioritize optimizing Transformer-based models like TrOCR for edge devices. Techniques such as pruning, quantization-aware training, and parameter reduction could significantly reduce model size and inference time while preserving performance. Testing these optimizations on actual devices will further validate their effectiveness.

REFERENCES

[1] D. Gifu, "AI-backed OCR in Healthcare," *Procedia Computer Science*, vol. 207, pp. 1134-1143, 2022. Available: https://doi.org/10.1016/j.procs.2022.09.169...

[2] Y. Li, "Synergizing Optical Character Recognition: A Comparative Analysis and Integration of Tesseract, Keras, Paddle, and Azure OCR," M.S. thesis, School of Computer Science, University of Sydney, Sydney, Australia, Feb. 2024.

[3] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," *arXiv preprint arXiv:2010.08678*, 2020. doi: 10.48550/arXiv.2010.08678.

[4] J. Pandey and A. R. Asati, "Lightweight convolutional neural network architecture implementation using TensorFlow lite," *International Journal of Information Technology*, vol. 15, pp. 2489–2498, 2023. doi: 10.1007/s41870-023-01320-9.

[5] S. Revanth and H. M. Roopa, "Handwritten Character Recognition," SSRN, June 2023. [Online]. Available: https://ssrn.com/abstract=4484321. doi: 10.2139/ssrn.4484321.

[6] G. Verma, Y. Gupta, A. M. Malik, and B. Chapman, "Performance evaluation of deep learning compilers for edge inference," in 2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2021, pp. 858–865.

[7] S. O. Bursa, O. D. Incel, and G. I. Alptekin, "Transforming deep learning models for resource-efficient activity recognition on mobile devices," in 2022 5th Conference on Cloud and Internet of Things (CIoT), 2022, vol. 62, pp. 83–89.

[8] I. Bolkisev and P. Ivanov, "Convolutional-Reccurrent Neural Network Adaptation for Real-time Text Recognition on CPU," *2023 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT)*, Yekaterinburg, Russian Federation, 2023, pp. 251-253, doi: 10.1109/USBEREIT58508.2023.10158839.

[9] IBM Corporation. (2023). Quantization. IBM Think. [Online]. Available: https://www.ibm.com/think/topics/quantization. [Accessed: 26-Sep-2024].

[10] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, "PP-OCR: A Practical Ultra Lightweight OCR System," *arXiv preprint arXiv:2009.09941*, 2020. [Online]. Available: https://arxiv.org/abs/2009.09941.

[11] Microsoft, "TrOCR Small Printed," Hugging Face, Accessed: Sep. 12, 2024. [Online]. Available: https://huggingface.co/microsoft/trocr-small-printed.

[12] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *arXiv preprint arXiv:2010.11929*, 2021. [Online]. Available: https://arxiv.org/abs/2010.11929.

[13] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," *arXiv preprint arXiv:2012.12877*, 2021. Available: https://arxiv.org/abs/2012.12877.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint arXiv:1810.04805*, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805.

[15] M. Li, T. Lv, C. Jingye, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models," *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, pp. 13094-13102, 2023, doi: 10.1609/aaai.v37i11.26538.

[16] UltOCR, "UltOCR: MASTER-based OCR Model," PyPI, Accessed: Sep. 12, 2024. [Online]. Available: https://pypi.org/project/ultocr/.

[17] N. Lu, W. Yu, X. Qi, Y. Chen, P. Gong, R. Xiao, and X. Bai, "MASTER: Multi-aspect non-local network for scene text recognition," *Pattern Recognition*, vol. 117, p. 107980, Sep. 2021, doi: 10.1016/j.patcog.2021.107980.

[18] V. I. Levenshtein, ''Binary codes capable of correcting deletions, insertions, and reversals,'' Soviet Physics Doklady, USSR, Kremlin Senate, Moscow, Tech. Rep. 8, 1966.

[19] "Character Error Rate," Hugging Face Documentation, 2022. [Online]. Available: https://huggingface.co/spaces/evaluate-metric/cer. [Accessed: Sep. 20, 2024].

[20] A. C. Morris, V. Maier, and P. D. Green, "From WER and RIL to MER and WIL: improved evaluation measures for connected speech recognition.," in *INTERSPEECH*, 2004, pp. 2765–2768.

[21] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 855-868, May 2009, doi: 10.1109/TPAMI.2008.137.

[22] "The IIIT 5K-word dataset," *Iiit.ac.in*. [Online]. Available: https://cvit.iiit.ac.in/research/projects/cvit-projects/the-iiit-5k-word-dataset. [Accessed: Sep. 14, 2024].

[23] RoboFlow, "Text Detection Box Dataset," Universe.RoboFlow.com. [Online]. Available:https://universe.roboflow.com/essamhaider/text_detection_box/dataset/1. [Accessed: 15-Sep-2024].